

Министерство науки и высшего образования Российской Федерации

НАЗВАНИЕ УНИВЕРСИТЕТА

Институт / факультет

Кафедра _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки _____

(наименование направления / специальности)

Разработка программного комплекса для централизованного управления мультисервисными Docker Compose-проектами (платформа Chronos)

Выполнил:

студент группы _____

(подпись, дата)

(Ф. И. О.)

Руководитель:

(подпись, дата)

(Ф. И. О., учёная степень, звание)

Город 2026

Содержание

1	Введение	1
1.1	Практическая мотивация	1
1.2	Идея Chronos	2
1.3	Цель и задачи работы	2
1.4	Объект, предмет и методы	3
1.5	Научная новизна и практическая значимость	3
1.6	Структура работы	3
2	Анализ предметной области и постановка задачи	4
2.1	Классы решений для контейнерной оркестрации	5
2.2	Сравнительный анализ Chronos, Kubernetes и Portainer	5
2.3	Обзор источников и нормативной базы	7
2.4	Позиционирование Chronos	8
2.5	Проблемы исходного сценария Compose-only	8
2.6	Функциональные требования	9
2.7	Нефункциональные требования	9
2.8	Выбор технологического стека	9
2.9	Краткая схема модулей (для последующих глав)	10
2.10	Постановка задачи	10
2.11	Цели, показатели качества и границы MVP	10
3	Архитектура и проектирование платформы Chronos	11
3.1	Три ключевых проекта репозитория	12
3.2	Общая схема взаимодействия и публикация	13
3.3	Стратегии резервного копирования томов	14
3.4	Маршрутизация TCP и роль Master	16
3.5	Работа агента с контейнерами и связь с Master	16
3.6	Декларативные проверки и артефакты деплоя	18
3.7	Кодовые интеграционные тесты ([Test]) и периодические job- ы ([Job])	18
3.8	Связка тестов, job-ов и пользовательского интерфейса	20

3.9	Модуль Chronos.Core: Fluent API, YAML и обратная реконструкция	20
3.10	Модуль Chronos.Agent: исполнение и тома	22
3.11	Модуль Chronos.Master: данные, UI и HAProxy	22
3.12	Сущности данных и связи	22
3.13	Наблюдаемость: Loki и Grafana	23
3.14	Безопасность, модель угроз и ограничения MVP	24
3.15	Выводы по главе	24
4	Реализация программной системы	24
4.1	Общие принципы реализации	25
4.2	Точка входа Master	25
4.3	Точка входа Agent	25
4.4	Реализация подсистемы TCP-маршрутизации	25
4.5	Алгоритм реестра TCP-маршрутов	26
4.6	Фоновая репликация томов	26
4.7	Проксирование запросов UI к агентам	26
4.8	Сводная таблица групп UI API	27
4.9	Контейнеризация стенда	27
4.10	Клиентская часть	27
4.11	Экраны веб-интерфейса Chronos.Master.Web	28
4.12	Практическая часть: запуск из кода	28
4.13	Безопасность	32
5	Экспериментальная проверка и оценка результатов	32
5.1	Методика испытаний	33
5.2	Критерии успешности	33
5.3	Результаты функциональных испытаний	34
5.4	Ограничения эксперимента	35
5.5	Обсуждение результатов	35
5.6	Направления дальнейшего развития	35
6	Заключение	36
A	Приложение A. Фрагменты конфигураций	37
A.1	Фрагмент docker-compose стенда	37

A.2	Фрагмент алгоритма выбора listen-порта	37
В	Приложение Б. Каталог сценариев и эксплуатационные материа-	
	лы	37
В.1	Каталог функциональных сценариев	38
В.2	Набор эксплуатационных рисков и мер снижения	39
В.3	Расширенный шаблон экспериментальной главы	40
В.4	Таблица API-эндпоинтов для документирования	41

1 Введение

Контейнеризация стала стандартом поставки серверного программного обеспечения: образы дают воспроизводимую среду, а декларативные манифесты снижают дрейф между стендами. Вместе с тем инженеру, который ведёт несколько сервисов на собственных или арендованных машинах, не всегда нужен полный масштаб промышленного оркестратора. Именно из практической потребности «управлять инфраструктурой так же предсказуемо, как кодом приложения», но без избыточной сложности, и возник проект **Chronos**.

1.1 Практическая мотивация

В экосистеме Docker давно существуют зрелые инструменты. **Kubernetes** даёт мощную модель деплоя, сетевых политик, масштабирования и самовосстановления, однако требует отдельного кластера, специалистов по эксплуатации и существенных накладных расходов на обучение и сопровождение. Для небольшой команды или личного стенда это часто избыточно: стоимость владения оказывается выше, чем выигрыш от функций, которые фактически не используются.

Portainer и схожие панели закрывают другой класс задач: наглядное управление контейнерами и compose-стеками через веб-интерфейс. При этом конфигурация по-прежнему в основном живёт в YAML-файлах и ручных действиях оператора; связка «репозиторий приложения — декларативное описание инфраструктуры — CI/CD» не всегда выражена единым программным API, которым удобно пользоваться из кода сборки или интеграционных тестов.

Опыт облачных платформ, в частности подходов, близких к экосистеме **Microsoft Azure** (инфраструктура и параметры приложения как управляемый из кода артефакт с проверкой на этапе сборки), показал привлекательность модели: разработчик описывает среду рядом с логикой сервиса, версионирует изменения тем же процессом, что и код, и может прогонять проверки до выката. Перенести эту идею в мир Docker без обязательного перехода на Kubernetes естественно через **Docker Compose**: формат широко распространён, хорошо документирован и поддерживается локально и в CI.

1.2 Идея Chronos

Платформа Chronos объединяет три уровня:

- **Chronos.Core** — библиотека на .NET с Fluent API для описания compose-проектов, генерации YAML, валидации, локального и удалённого запуска; поддержка кодовых тестов и job-ов ([Test], [Job]), прикрепляемых к сервису через UseTests/UseJobs;
- **Chronos.Agent** — сервис на машине с Docker Engine, который принимает манифесты, выполняет docker compose, отдаёт статус, логи и снимки томов;
- **Chronos.Master** (и веб-клиент **Chronos.Master.Web**) — центральный узел: реестр агентов, аудит, проксирование запросов UI к агентам, политики резервного копирования томов в объектное хранилище, динамическая генерация TSP-конфигурации для HAProxy.

Таким образом, Chronos не заменяет Docker, а надстраивает над ним управляющий контур с сохранением compose-парадигмы.

1.3 Цель и задачи работы

Цель работы — разработать и описать программный комплекс Chronos для централизованного управления Docker Compose-проектами на нескольких узлах с поддержкой наблюдаемости, резервного копирования томов и динамической TSP-маршрутизации.

Задачи:

1. Проанализировать ограничения существующих классов решений (Kubernetes панели управления, «голый» Compose) для целевого сценария.
2. Сформулировать требования к модульной архитектуре (Core, Agent, Master) и к нефункциональным свойствам (наблюдаемость, воспроизводимость).
3. Спроектировать взаимодействие компонентов: публикация проектов, heartbeat агентов, политики бэкапа томов, реестр TSP-маршрутов HAProxy.

4. Реализовать серверные и клиентские части, развернуть контрольный стенд в Docker.
5. Провести функциональную проверку ключевых сценариев и зафиксировать ограничения MVP.

1.4 Объект, предмет и методы

Объект исследования — распределённая программная система управления контейнерными приложениями на базе Docker Compose.

Предмет исследования — методы построения агентной архитектуры, алгоритмы согласования состояния маршрутов и политик резервного копирования с объектным хранилищем S3-совместимого типа, а также интеграция с HAProxy и стеком Loki/Grafana.

Методы: системный анализ, объектно-ориентированное проектирование, экспериментальное развёртывание на локальном стенде, анализ логов и конфигураций.

1.5 Научная новизна и практическая значимость

К практической новизне относятся: связка «Fluent API ↔ YAML ↔ удалённый агент» с возможностью обратного разбора compose-файла в цепочку вызовов билдера; централизованные политики снимков томов с учётом свободного места на узле агента и ретенции копий в бакете; динамическое обновление фрагмента конфигурации HAProxy с учётом эксплуатационных деталей (кодировка UTF-8 без BOM, перезагрузка прокси).

Практическая значимость — готовый к использованию контур для локальной и небольшой командной инфраструктуры, снижающий долю ручных операций при публикации сервисов и при публикации TCP-портов наружу.

1.6 Структура работы

Работа состоит из введения, глав с анализом предметной области, архитектуры и проектирования, реализации, результатов испытаний, заключения,

приложений и списка использованных источников. В приложениях приведены вспомогательные таблицы сценариев и справочные материалы.

2 Анализ предметной области и постановка задачи

2.1 Классы решений для контейнерной оркестрации

В индустрии можно выделить несколько устойчивых классов средств.

Кластерные оркестраторы (прежде всего Kubernetes) ориентированы на крупные установки: декларативные манифесты, горизонтальное масштабирование, сетевые политики, хранилища и операторы. Недостатки для небольших проектов — высокая кривая обучения, необходимость сопровождения control plane, иной цикл выпуска по сравнению с «одним compose-файлом на сервис».

Панели управления Docker (в т.ч. Portainer) упрощают операции через UI и API поверх уже запущенного Docker. Они хорошо закрывают задачу визуализации и точечных действий, но реже выступают единой библиотекой для описания инфраструктуры из кода приложения или сборочного конвейера с тем же уровнем типобезопасности и переиспользования, что у доменной модели на языке разработки.

Легковесный сценарий Docker Compose сохраняет простоту и прозрачность: один файл описывает сервисы, сети и тома. Проблема возникает при нескольких хостах: нет встроенного центра реестра узлов, единого API публикации, согласованной политики бэкапа томов и управляемой публикации TCP-портов наружу без ручной правки прокси.

2.2 Сравнительный анализ Chronos, Kubernetes и Portainer

Таблица 1 обобщает отличия по наиболее значимым для данной работы критериям. Оценки носят качественный характер: конкретные редакции Portainer и дистрибутивов Kubernetes могут расширять отдельные пункты плагинами.

Таблица 1: Сравнение Chronos, Kubernetes и Portainer

Критерий	Chronos	Kubernetes	Portainer
Целевая аудитория	Команды и индивидуальные разработчики, уже использующие Compose; небольшие стенды.	Организации с выделенной платформенной командой и кластерами от нескольких узлов.	Администраторы и разработчики, управляющие Docker через UI/API без обязательной кластеризации.
Модель описания сервисов	Docker Compose (YAML) и/или Fluent API на C# в библиотеке Core.	Pod/Deployment/Service/Ingress и др.; часто Helm/Kustomize.	В основном UI и Docker Compose-стеки; «инфраструктура как код» чаще внешняя по отношению к Portainer.
Несколько хостов без K8s	Ядро: Master + набор Agent; единый UI и политики.	Требуется полноценный кластер Kubernetes.	Есть Portainer Agent на узлах; фокус на обзоре/действиях, не на единой модели публикации compose из .NET.
Программируемость из кода приложения	Прямое использование Chronos.Core в сборках, тестах, консольных утилитах.	Клиенты API (kubectl, client-go, CDK8s и т.д.); иная модель абстракций.	REST/API Portainer; нативной Fluent-модели compose под .NET нет.
Политики снимков томов в объектное хранилище	Встроены: политики на Master, снимок на Agent, S3-совместимый бакет, ретенция MaxCopies.	VolumeSnapshot API, внешние операторы бэкапа; настройка сложнее.	Есть сценарии резервного копирования томов (зависит от редакции); нет той же связки «политика Master → агент» из коробки.
Динамический TCP edge	Реестр маршрутов Master, генерация HAProxy, диапазон listen-портов.	Service type LoadBalancer / Ingress-контроллеры; своя модель.	Не является центральной функцией продукта; настраивается внешними средствами.

Продолжение на следующей странице

Таблица 1 — продолжение

Критерий	Chronos	Kubernetes	Portainer
Самописные про-верки и фоновые задачи в рантайме compose	Кодовые тесты ([Test]) и job-ы ([Job]) в подключаемых сборках; запуск на агенте после старта и по расписанию.	Probes (liveness/readiness), CronJob как отдельная абстракция, не «внутри» compose-сервиса в смысле Chronos.	Healthcheck в compose; произвольный C# на агенте в модели Chronos не предусмотрен.
Порог входа и сопровождение	Ниже, чем у production Kubernetes; нужны Docker и понимание compose.	Высокий; требуется экспертиза по сети, хранилищам, обновлениям control plane.	Средний для UI-операций; кластерные сценарии усложняются.
Отказоустойчивость control plane (типичный OSS-сценарий)	MVP: один Master; задел под лидерство.	Распределённый etcd, несколько control plane (в зависимости от установки).	Зависит от установки Portainer Server; обычно репликация/reverse проху настраиваются отдельно.

2.3 Обзор источников и нормативной базы

Для обоснования архитектурных решений и выбора стека опирались на открытые спецификации и документацию продуктов, а не на единичные статьи.

Docker Compose. Формат манифеста и семантика полей `services`, `networks`, `volumes`, `depends_on` описаны в официальной спецификации Compose; Chronos сознательно не дублирует планировщик контейнеров, а генерирует совместимый YAML и вызывает штатный `docker compose` на агенте. Это снижает риск расхождения с поведением инструментов, которые уже знают разработчики.

Kubernetes. Руководства по Deployment, Service, Ingress и VolumeSnapshot дают ориентиры по тому, какие задачи в крупных кластерах решаются декларативно; сопоставление с Chronos выполняется в таблице 1. Для ВКР важно зафиксировать: Kubernetes остаётся эталоном по полноте модели, но избыточен для сценария «несколько VPS с Docker» без выделенной платформенной команды.

HAProxy. Режим `mode tcp`, директивы `bind` и `server`, а также ограничения парсера конфигурации (кодировка, переносы строк) описаны в офици-

альной документации HAProxy. Динамическое подключение фрагмента конфигурации через `include` и сигнал перезагрузки — распространённый приём; в работе он применён для реестра маршрутов Chronos.

Portainer и смежные панели. Документация API и модели агентов Portainer иллюстрирует подход «центральный сервер + агенты на хостах»; Chronos перенимает идею регистрации узлов, но добавляет доменную модель `compose` в `.NET` и политики бэкапа томов, не претендуя на универсальную административную всего Docker-хоста.

Наблюдаемость. Модель логов Loki (лейблы, потоки, запросы LogQL) и роль Grafana как визуального слоя описаны в документации Grafana Labs. Выбор `push`-логов по HTTP упрощает развёртывание на малых стендах по сравнению с полноценным стеком на базе Prometheus для каждого узла.

2.4 Позиционирование Chronos

Chronos занимает нишу между «только Compose на каждой машине» и полноценным Kubernetes: сохраняется формат и инструментарий Compose, добавляется программируемое описание через `.NET`-библиотеку, центральный узел и агенты на хостах с Docker.

Вдохновляющим примером для постановки задачи послужили облачные практики (в духе Azure Resource Manager / Bicep и родственных подходов), где параметры среды задаются декларативно и проверяются до развёртывания. В Chronos аналогичную роль выполняет связка Fluent API и валидатора с последующей генерацией YAML и выкладкой на агент.

2.5 Проблемы исходного сценария Compose-only

Без управляющего слоя типичны следующие трудности:

- ручной или нестандартизованный деплой на разных хостах;
- отсутствие единого API и журнала операций;
- сложность маршрутизации внешнего TCP-трафика к сервисам на разных машинах;
- отсутствие единой политики снимков томов и хранения бэкапов в объектном хранилище.

2.6 Функциональные требования

1. Описание compose-проекта из кода (Fluent API) и/или YAML с валидацией.
2. Публикация проекта на удалённый агент по HTTP, управление жизненным циклом (start/stop/restart).
3. Регистрация агентов на Master, heartbeat, отображение состояния в UI.
4. Декларативные проверки и задания на стороне агента (после старта и по расписанию).
5. Резервное копирование томов по политикам Master с выгрузкой в S3-совместимое хранилище.
6. Динамическое добавление и удаление TCP-маршрутов с генерацией конфигурации HAProxy.
7. Централизованный сбор логов приложений в Loki и визуализация в Grafana.

2.7 Нефункциональные требования

- воспроизводимость поведения в локальной и контейнерной среде;
- расширяемость (новые сценарии без переписывания ядра);
- наблюдаемость (логи, метрики, трассировка запросов к API);
- устойчивость к рестартам в модели single-master MVP;
- контроль секретов и редактирование чувствительных фрагментов в логах.

2.8 Выбор технологического стека

- **.NET 8** — единая платформа для Core, Agent и Master;
- **Entity Framework Core + PostgreSQL** — метаданные кластера, аудит, политики бэкапа;

- **React + Vite + TypeScript** — веб-интерфейс;
- **HAProxy** — TCP-прокси с динамически подключаемым фрагментом конфигурации;
- **Loki + Grafana** — операционные логи;
- **S3 API (MinIO и др.)** — хранение снимков томов.

2.9 Краткая схема модулей (для последующих глав)

Для дальнейшего изложения зафиксируем декомпозицию:

1. **Chronos.Core** — доменная модель compose, генерация и разбор YAML, клиент агента, локальный тестер, артефакты деплоя, исполнение кодовых jobs и тестов.
2. **Chronos.Agent** — исполнение docker compose, хранение проектов, снимки томов, планировщик проверок, связь с Master.
3. **Chronos.Master** — персистентность, UI API, прокси к агентам, HAProxy-реестр, фоновая репликация томов, раздача собранного SPA.

2.10 Постановка задачи

Требуется спроектировать и реализовать программный комплекс, который принимает декларативное описание контейнерных сервисов (из кода или YAML), публикует его на удалённых агентных узлах, обеспечивает централизованное управление запуском, наблюдаемость, политики резервного копирования томов и проксирование внешнего TCP-трафика к сервисам на хостах агентов через HAProxy, сохраняя совместимость с Docker Compose.

2.11 Цели, показатели качества и границы MVP

Цель разработки сформулирована во введении: получить воспроизводимый контур управления Compose-проектами на нескольких узлах с веб-интерфейсом и минимально достаточной наблюдаемостью.

Показатели качества (в качественной форме, пригодной для проверки на стенде):

- *Корректность API*: ответы согласованы с фактическим состоянием агента и БД; ошибки валидации возвращаются до побочных эффектов (например, до записи конфликтующего TSP-маршрута).
- *Надёжность сценария TSP*: после обновления `chronos-tsp.cfg` и перезагрузки HAProxy внешнее TSP-соединение доходит до опубликованного на хосте агента порта в типовой конфигурации Docker Desktop / Linux.
- *Воспроизводимость бэкапа*: политика на Master инициирует снимок не чаще заданного интервала; число объектов в бакете не превышает `MaxCopies` после процедуры `prune`.
- *Наблюдаемость*: критические ошибки API и фоновых служб видны в Loki и позволяют локализовать узел (Master или Agent) по лейблам.

Границы MVP (что сознательно не входит в первую версию, но оставляет направление развития):

- отказоустойчивый кластер Master и автоматический failover;
- полноценный RBAC и интеграция с корпоративными IdP;
- автоматическое обнаружение публикуемых портов compose без участия оператора при создании TSP-маршрута;
- формальные нагрузочные тесты с отчётом SLA (рекомендуются как отдельный этап после защиты прототипа).

3 Архитектура и проектирование платформы Chronos

Настоящая глава даёт целостное описание архитектуры: от декомпозиции на три основных проекта до подсистем маршрутизации, резервного копирования томов и наблюдаемости. Детали реализации API и развёртывания раскрываются в следующей главе.

3.1 Три ключевых проекта репозитория

Исходный код Chronos организован в виде нескольких артефактов сборки; для архитектурного анализа выделяют три несущих столпа.

Chronos.Core — библиотека (.NET), не привязанная к конкретному хосту. Она инкапсулирует доменную модель Docker Compose (сервисы, сети, тома, секреты, расширения), предоставляет Fluent API для пошагового построения конфигурации, генерацию YAML, best-effort разбор существующего `docker-compose.yml` обратно в билдер, валидацию, локальный запуск через CLI Docker Compose, а также HTTP-клиент для публикации и управления проектами на удалённом агенте. Сюда же относятся вспомогательные подсистемы: упаковка *артефактов деплоя* (файлы и каталоги, включаемые в архив при выкладке), декларативные проверки, исполнение кодовых jobs и тестов, ограничения безопасности для выполняемых команд.

Chronos.Agent — контейнеризированное серверное приложение (ASP.NET Core), разворачиваемое на машине с Docker Engine. Оно имеет доступ к `docker.sock` (или эквивалентному API), хранит каталог опубликованных проектов, по запросу выполняет `docker compose up/down`, отдаёт статус контейнеров и потоки логов, формирует архивы и потоковые снимки томов, обрабатывает фоновый планировщик проверок и при старте регистрируется на Master с периодическим heartbeat.

Chronos.Master — центральный узел кластера: хранит состояние в PostgreSQL (агенты, аудит, размещение проектов, политики и состояние бэкапов томов, архивные проекты и др.), отдаёт REST API для агентов и для UI, проксирует запросы интерфейса к агентам, опционально ведёт реестр TCP-маршрутов и генерирует фрагмент конфигурации для HAProxy. Клиентская часть **Chronos.Master** (React) собирается в статику и раздаётся Master из `wwwroot`.

Четвёртый модуль, **Chronos.Master.Web**, архитектурно является фронт-

тендом Master и в дальнейшем при упоминании «Master» в контексте UI/API подразумевается связка backend+SPA.

3.2 Общая схема взаимодействия и публикация

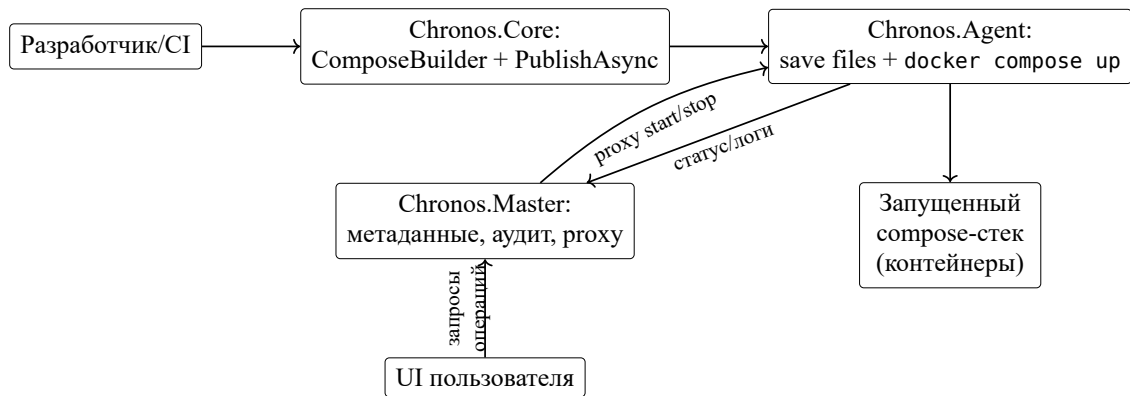


Рис. 3.1. Общий процесс публикации и управления проектом через Chronos

Типовой поток данных следующий.

1. Разработчик или CI формирует описание проекта в коде через `ComposeBuilder` либо готовит YAML вручную.
2. Операция публикации (`PublishAsync` и родственные методы в Core) упаковывает compose-файл и артефакты деплоя в передаваемый на агент архив и отправляет HTTP-запрос на Agent.
3. Agent сохраняет файлы в своём рабочем каталоге, при необходимости обновляет метаданные в локальной БД агента.
4. Команды жизненного цикла снова идут через HTTP к Agent; Master может участвовать в сценарии «кластерной» публикации, маршрутизируя вызовы к нужному агенту.
5. UI пользователя обращается к Master; запросы, требующие данных с конкретного узла (логи, граф сервисов), проксируются Master к соответствующему Agent.

Таким образом, Master не заменяет Docker на узле: он координирует метаданные и политики, а исполнение остаётся на Agent.

3.3 Стратегии резервного копирования томов

Подсистема бэкапа томов строится вокруг трёх элементов: *политика* на Master, *исполнение снимка* на Agent и *объектное хранилище* S3-совместимого типа (в т.ч. MinIO).

Политика задаёт имя проекта, шаблон имени тома (VolumePatternMatcher), максимальное число успешных копий в бакете (MaxCopies), признак включения и опциональный дополнительный префикс ключей. Состояние последнего бэкапа (время, оценка объёма) хранится в БД Master для адаптации интервала и проверки диска.

Фоновый сервис VolumeReplicationHostedService на стороне лидирующего экземпляра Master периодически обходит включённые политики. Перед запуском снимка запрашиваются метрики свободного места на диске агента; при недостатке места копия пропускается. По префиксу ключей в бакете выполняется листинг существующих объектов; лишние версии удаляются согласно MaxCopies. Решение о новом снимке учитывает интервал политики и размер предыдущей передачи.

Agent предоставляет эндпоинт снимка тома в объектное хранилище: запускается процесс архивации данных Docker-тома проекта (с поддержкой сжатия gzip), поток stdout направляется в загрузчик S3 (VolumeTarS3Uploader). Альтернативно доступны сценарии выдачи tar потоком клиенту и загрузки по предписанному URL — это покрывает кастомные пайплайны.

Итоговая стратегия — *централизованно иницилируемые, дедуплицируемые по числу копий снимки с проверкой ресурсов узла*, без блокировки основного API на длительное время за счёт асинхронности фоновой задачи Master.

Рисунок 3.2 показывает основной поток: решение о запуске принимает Master (с учётом политики, состояния и диска), исполнение снимка и загрузка байтов выполняются на агенте, метаданные копий хранятся в S3-совместимом бакете. Пунктирные альтернативы (выдача tar клиенту, загрузка по внешнему presigned URL) в схеме не показаны, но поддерживаются отдельными endpoint-ами агента.

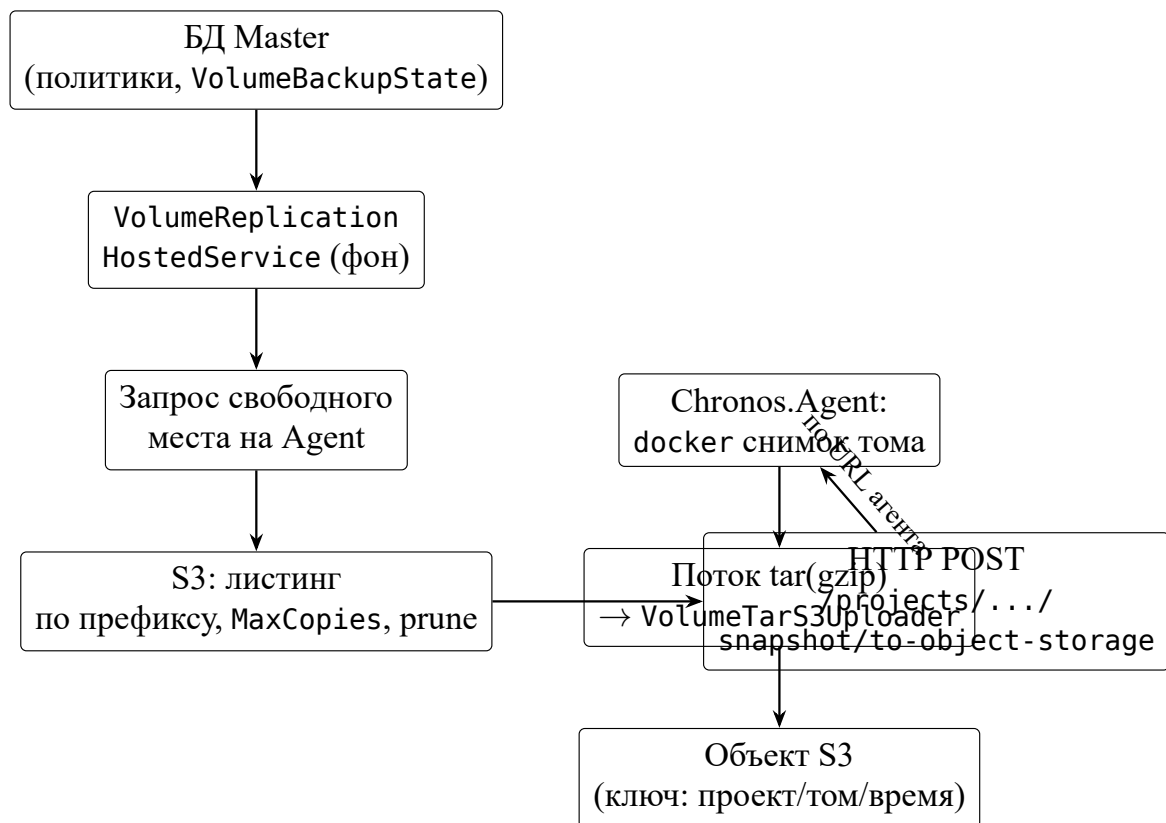


Рис. 3.2. Взаимодействие компонентов при резервном копировании тома в объектное хранилище

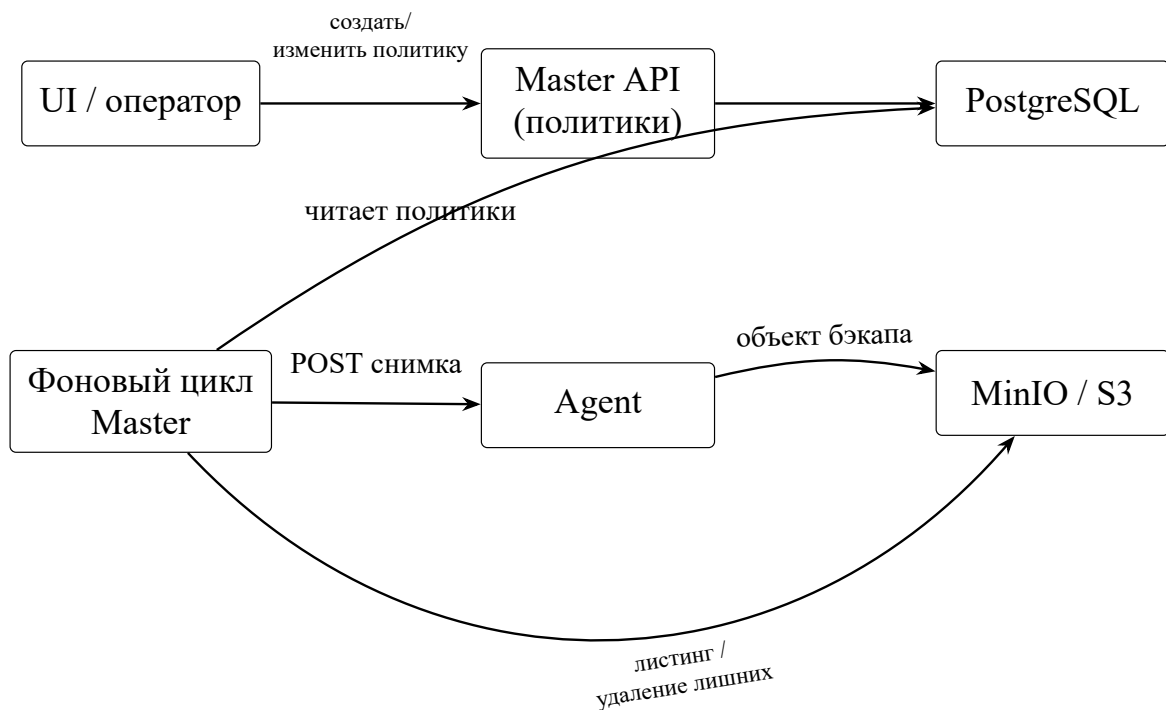


Рис. 3.3. Роли узлов при управляемом бэкапе: оператор и UI, Master (политика и оркестрация), Agent (снимок), хранилище

3.4 Маршрутизация TCP и роль Master

Внешние клиенты, подключающиеся по TCP к опубликованным сервисам на хостах агентов, не должны зависеть от внутренней топологии Docker. В Chronos для этого используется HAProxy: Master ведёт реестр маршрутов (`INaproxyTcpRouteRegistry`), для каждого маршрута задаются listen-порт (или автоматический выбор из разрешённого диапазона) и адрес backend (часто `host.docker.internal:PORT` в bridge-сценарии Docker Desktop).

Процедура добавления маршрута включает валидацию портов, проверку конфликтов, сохранение JSON-реестра и генерацию текстового фрагмента `chronos-tcp.cfg`. Отдельный shell-скрипт в compose-стенде отслеживает изменение файла и посылает HAProxy сигнал перезагрузки, что избавляет оператора от ручного `SIGHUP`.

Если переменная окружения с каталогом динамической конфигурации не задана, реестр заменяется заглушкой: остальная функциональность кластера сохраняется.

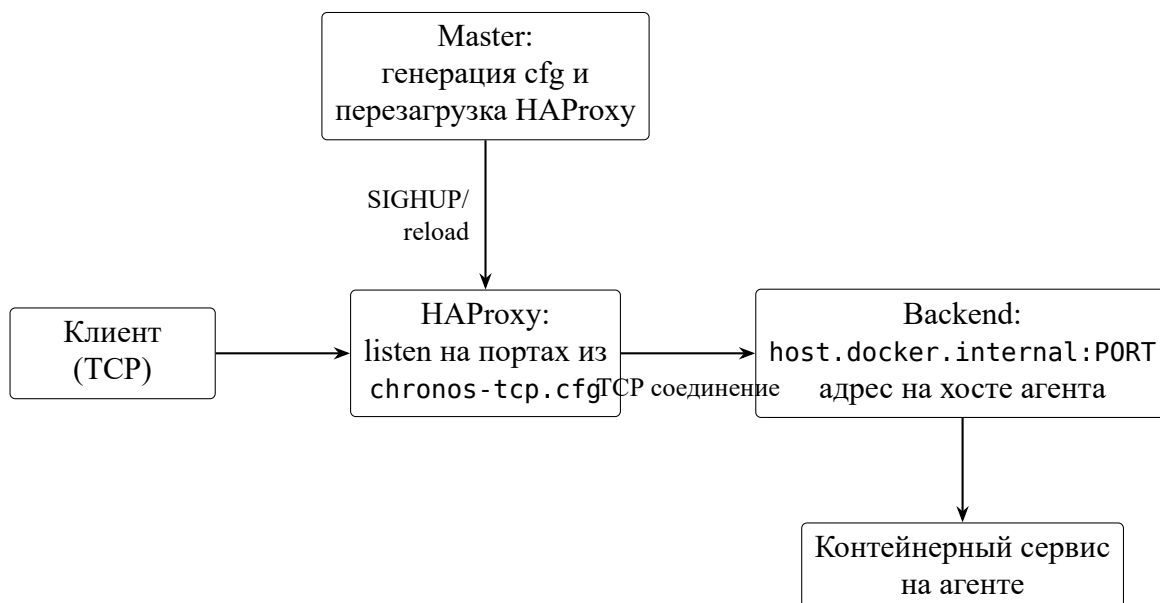


Рис. 3.4. Работа TCP-маршрутизации: Master управляет конфигурацией HAProxy, клиент подключается к listen, трафик уходит на backend агента

3.5 Работа агента с контейнерами и связь с Master

Agent не «подменяет» собой Docker Scheduler: он вызывает штатный `docker compose` с параметрами проекта. Отслеживание состояния выполня-

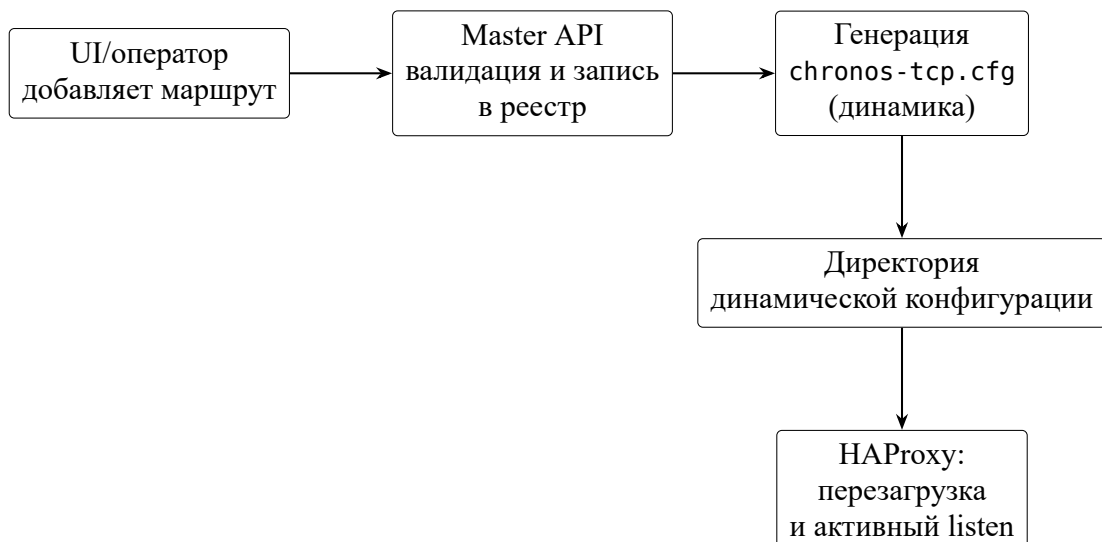


Рис. 3.5. Добавление TCP-маршрута: UI вызывает Master, Master обновляет динамический конфиг и инициирует reload HAProxy

ется через команды статуса и через пользовательские *декларативные проверки* в манифесте: часть из них может выполняться сразу после старта (OnStartup), часть — по интервалу в фоновом планировщике (SchedulerHostedService). Критичные проваленные проверки могут инициировать перезапуск сервиса, что задаёт политику самовосстановления на уровне compose-сервиса.

С Master агент связывается при старте (регистрация) и далее heartbeat-сообщениями; Master помечает узлы как активные или устаревшие по TTL. Это основа для отображения топологии в UI и для выбора URL агента при политиках бэкапа.

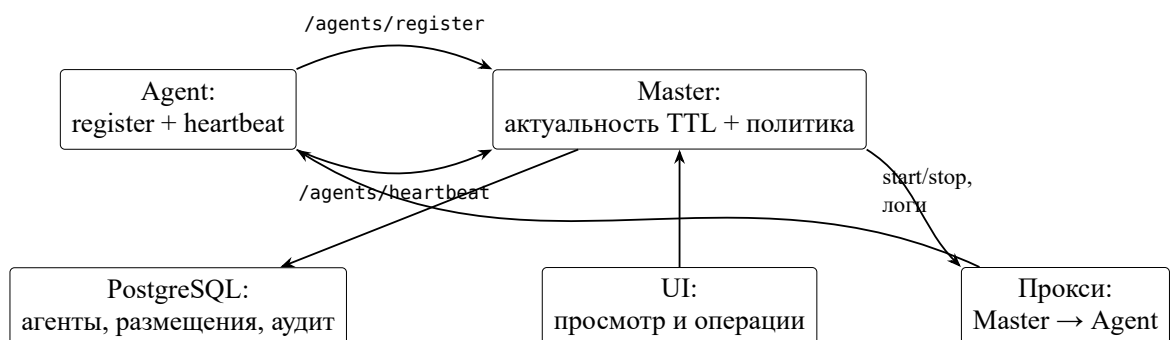


Рис. 3.6. Взаимодействие Master и Agent: регистрация/heartbeat и проксирование операций

3.6 Декларативные проверки и артефакты деплоя

Декларативные проверки задаются в манифесте сервиса (например, HTTP-проверка, ожидание порта) и выполняются на агенте через `DeclarativeCheck`. Часть сценариев запускается сразу после старта `compose`, часть — по расписанию в `SchedulerHostedService`. У проверки есть уровень критичности: при провале критичной проверки возможен перезапуск сервиса.

Артефакты деплоя (`DeployArtifact`) позволяют включить в публикуемый пакет файлы и каталоги с машины сборки (тестовые данные, скрипты, результаты CI). При публикации они упаковываются в `tar` вместе с `compose` (`DeployArtifactTarWriter`), поэтому на агенте проект оказывается самодостаточным.

3.7 Кодовые интеграционные тесты ([Test]) и периодические job-ы ([Job])

Важная особенность Chronos — возможность **прикрепить собственные тесты и фоновые задания непосредственно к compose-сервису**, который разворачивается на агенте. Логика живёт в отдельной .NET-сборке (или в том же решении), публикуется вместе с артефактами деплоя и указывается в Fluent-описании сервиса, поэтому проверки версионизируются в Git рядом с приложением.

Связка с ServiceBuilder. В цепочке описания сервиса вызываются методы `UseTests<TTests>() / UseTests(params Type[])` и `UseJobs<TJobs>() / UseJobs(params Type[])` (см. `ServiceBuilder` в `Chronos.Core`): для каждого переданного типа рефлексией сканируются методы с атрибутами `[Test]` и `[Job]`, в модель сервиса добавляются записи `CodeTestEntry` и `CodeJobEntry` с путём к сборке относительно корня публикуемого проекта. После `PublishAsync` агент получает и `compose`, и DLL, и манифест с перечислением тестов/job-ов для данного сервиса.

Отдельная подсистема `Chronos.Core` позволяет автору инфраструктуры написать *обычный C#-код*, который выполняется уже на агенте в контексте развёрнутого проекта: с доступом к рабочему каталогу `compose`, имени проекта и *имени сервиса*, HTTP-клиенту и обёрткам над `docker compose/docker exec`.

Кодовые тесты. Метод помечается атрибутом [Test] (тип TestAttribute в пространстве имён Chronos.Core). Допустимая сигнатура — Task или Task<CodeTestOutcome>. Параметры метода — только ComposeTestContext и/или CancellationToken. В атрибуте задаются: строковый Id, критичность (Critical/Warning), признак OnStartup, опциональный IntervalMinutes для повторных запусков, Order (меньше — раньше в очереди). Контекст ComposeTestContext даёт ExecInServiceAsync (команда внутри контейнера выбранного сервиса) и RunComposeAsync (вызов compose с хоста агента).

На агенте CodeTestRunner загружает сборку по пути из манифеста, находит тип и метод, при необходимости создаёт экземпляр класса, вызывает метод и учитывает критичность при решении о перезапуске сервиса.

Листинг 1: Пример кодового теста с атрибутом [Test] (сканирование и вызов на агенте)

```
using Chronos.Core;

namespace SampleTests;

/// <summary>Пример кодового теста: HTTP с хоста, где крутится compose (
    LocalTester / агент).</summary>
public sealed class NginxTest
{
    [Test]
    public async Task<CodeTestOutcome> RespondsOn8080(ComposeTestContext ctx,
        CancellationToken cancellationToken = default)
    {
        using var response = await ctx.Http.GetAsync("http://127.0.0.1:8080/",
            cancellationToken).ConfigureAwait(false);
        if (!response.IsSuccessStatusCode)
            return CodeTestOutcome.Fail($"HTTP {(int)response.StatusCode}");

        return CodeTestOutcome.Ok();
    }
}
```

Периодические job-ы. Методы с атрибутом [Job] (JobAttribute) оформляются так же, с контекстом ComposeJobContext и возвратом Task / Task<CodeJobOutcome>. Типичное применение — периодическая обслуживающая работа внутри уже запущенного стека (ротация логов, фоновая синхронизация), а не замена внеш-

нему cron на хосте. Исполнение — CodeJobRunner.

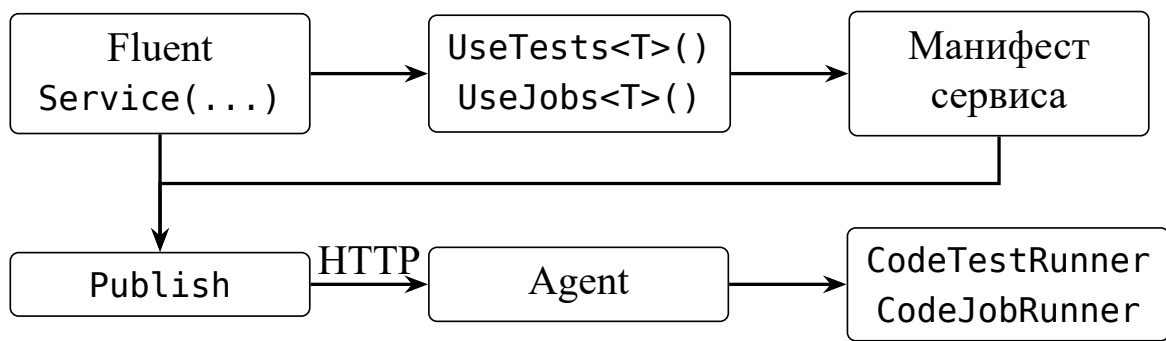


Рис. 3.7. Прикрепление сборок с [Test] / [Job] к сервису и исполнение на агенте

Рисунок 3.7 подчёркивает, что тесты и job-ы не «внешний скрипт», а часть описания сервиса, доставляемая на узел в одном пакете с compose.

Таким образом, разработчик может выразить сложную проверку или обслуживающую задачу программно, не ограничиваясь декларативными проверками YAML. Результаты последних запусков агент добавляет в диагностический JSON; Master отдаёт их в UI (рис. 4.4 в главе реализации).

3.8 Связка тестов, job-ов и пользовательского интерфейса

На стороне ProjectDetailPage (React) список recentTests и recentJobs разбирается из диагностики и показывается в виде таблицы с типом записи (test/job), идентификатором, сообщением и временем. Это снимает необходимость вручную подключаться к хосту агента для первичной диагностики после деплоя.

3.9 Модуль Chronos.Core: Fluent API, YAML и обратная реконструкция

Центральный тип — ComposeBuilder. Цепочка вызовов вида new ComposeBuilder накапливает объекты доменной модели и в конце может сгенерировать строку YAML (GenerateYaml) или сохранить файл.

Листинг 2: Пример сборки compose-проекта через Fluent API ComposeBuilder

```
var compose = new ComposeBuilder()
    .WithProjectName("chronos-mvp-test")
```

```

        .AddNetwork("backend")
        .AddVolume("pg_data")
        .AddVolume("redis_data")

        .AddService(s => s
            .WithName("postgres")
            .UseImage("postgres:16-alpine")
            .AddEnvironment("POSTGRES_USER", "chronos")
            .AddEnvironment("POSTGRES_PASSWORD", "chronos")
            .AddEnvironment("POSTGRES_DB", "chronos_app")
            .AddPort(55432, 5432)
            .AddVolume("pg_data", "/var/lib/postgresql/data")
            .DependsOn())

        .AddService(s => s
            .WithName("redis")
            .UseImage("redis:7-alpine")
            .AddPort(56379, 6379)
            .AddVolume("redis_data", "/data"))

        .AddService(s => s
            .WithName("adminer")
            .UseImage("adminer:4")
            .AddPort(58080, 8080)
            .DependsOn("postgres"))

        .AddService(s => s
            .WithName("nginx")
            .UseImage("nginx:alpine")
            .AddPort(58081, 80)
            .DependsOn("postgres", "redis"));

var composePublisher = compose.Build();

```

Разбор существующего compose. Класс `ComposeYamlParser` выполняет разбор YAML в объекты билдера (подход best-effort: не каждое поле спецификации `Compose` может быть отражено в модели). Методы `ComposeBuilder.FromYaml` и `LoadFromAgentAsync` позволяют импортировать файл с диска или непосредственно с агента, что поддерживает сценарий «подтянуть то, что уже развернуто, и продолжить правки в коде».

Реконструкция Fluent-кода. Для отладки и round-trip в билдере реализована генерация текста вызовов API по текущему состоянию модели (см. комментарии в ComposeBuilder к реконструкции): это облегчает сравнение исходного YAML и эквивалентного программного описания.

Удалённое управление. Тип `HttpDeployAgent` инкапсулирует HTTP-контракт агента: публикация, старт/стоп, список проектов, получение compose. Высокоуровневые методы билдера связывают генерацию YAML с этими вызовами.

3.10 Модуль Chronos.Agent: исполнение и тома

Agent конфигурируется путём к корню приложений, параметрами Docker Compose CLI, URL Master и ключами API. Основные группы endpoint-ов: проекты (список, загрузка/скачивание compose, жизненный цикл), логи, тома и снимки, служебная информация для Master.

Снимок тома использует блокировку на имя тома, чтобы не допустить параллельных несогласованных чтений. Режим `to-object-storage` использует учётные данные объектного хранилища из конфигурации агента; при отсутствии конфигурации возвращается отказ с пояснением.

3.11 Модуль Chronos.Master: данные, UI и HAProxy

Master использует `ChronosMasterDbContext` (Entity Framework Core) и PostgreSQL как единственный источник правды для метаданных кластера. API разделено на машинно-ориентированные контракты для агентов и UI-версию (`UiV1Extensions`), а также опциональную песочницу для исполнения Fluent-сценариев без записи в производственные данные.

Генерация HAProxy описана выше; важно подчеркнуть разделение: Master не принимает внешний TCP-трафик сам по себе в типовом стенде — он лишь обновляет конфигурацию, а HAProxy слушает порты.

3.12 Сущности данных и связи

Ниже перечислены основные группы сущностей, на которых держится логика Master; точные имена таблиц и полей определяются миграциями EF Core, здесь зафиксирована семантика.

Агенты и доступность. Регистрация агента фиксирует идентификатор, базовый URL и время последнего heartbeat. По TTL неактивные записи могут удаляться фоновой задачей: это влияет на выбор URL при политиках бэкапа и на отображение топологии в UI.

Проекты и размещение. Размещение (*placement*) связывает логический проект с конкретным агентом: где лежит каталог compose, куда направлять команды жизненного цикла. Архив проектов позволяет хранить метаданные снятых с эксплуатации конфигураций без удаления истории.

Аудит. Операции, инициированные пользователем или API (публикация, изменение маршрутов, удаление), фиксируются в журнале аудита с временной меткой и инициатором. Это снижает неопределённость при разборе инцидентов «кто и когда изменил маршрут».

Политики и состояние бэкапа томов. Политика описывает проект, шаблон имени тома, интервал, MaxCopies и признак активности. Состояние последнего успешного снимка хранится для адаптации расписания и проверки диска на агенте перед новым запуском.

ТСР-маршруты. Реестр маршрутов хранит listen-порт (или признак автоматического выбора), целевой backend (хост и порт на стороне агента), идентификаторы для генерации фрагмента `chronos-tcr.cfg`. Консистентность с файлом на диске обеспечивается атомарной записью и внешним reload Nginx.

3.13 Наблюдаемость: Loki и Grafana

Компоненты Master и Agent настраиваются на отправку структурированных логов по HTTP в Loki (модуль `LokiHttpLogging`). На стенде из `docker-compose` поднимаются Loki и Grafana; преднастроенные панели облегчают поиск ошибок API, сбоях фоновых служб и повторяющихся предупреждений при публикации.

Лейблы и поиск. Рекомендуется задавать лейблы уровня приложения (`app`, `instance`, `environment`), чтобы в LogQL отфильтровать поток: например, запрос вида `{app="chronos-master"}` ограничивает выборку логами мастер-узла; добавление условия по уровню (`| json | level="Error"`) ускоряет разбор инцидентов после деплоя.

Ограничения подхода. Loki не заменяет трассировку распределённых запросов: связка «запрос UI → прокси Master → Agent» по одному trace-id в

MVP не сквозная. Для дипломного стенда достаточно корреляции по времени и по идентификаторам в тексте сообщения; развитие — в сторону OpenTelemetry при появлении требований.

3.14 Безопасность, модель угроз и ограничения MVP

Контур доверия. Агент доверяет Master при регистрации и heartbeat; Master доверяет агенту при проксировании запросов UI, если выбранный агент авторизован ключом API. Внешний клиент TCP не аутентифицируется Chronos на уровне приложения — граница безопасности для публикуемых портов задаётся сетевыми ACL и политикой организации.

Секреты и логи. Пароли подключения к БД и ключи API не должны попадать в stdout в открытом виде; в коде предусмотрено редактирование чувствительных подстрок при логировании. Полная интеграция с Vault или аналогами в объёме работы не реализована — секреты задаются переменными окружения и файлами конфигурации стенда.

Исполнение на агенте. Доступ агента к `docker.sock` эквивалентен высоким привилегиям на хосте; снижение риска достигается ограничением списка допустимых команд и сетевой изоляцией узла, а не заменой модели Docker.

Ограничения версии. Single-master без горячего failover; упрощённая авторизация по сравнению с корпоративным RBAC; корректность TCP-маршрутов зависит от согласованности `host/container` портов и режима сети Docker (в частности, доступность `host.docker.internal` на стороне NProxy).

3.15 Выводы по главе

Спроектированная архитектура разделяет описание среды (Core), исполнение (Agent) и координацию (Master), добавляя сквозные подсистемы маршрутизации, бэкапа томов и наблюдаемости. Такая декомпозиция соответствует поставленным требованиям и подготавливает читателя к описанию реализации и испытаний.

4 Реализация программной системы

4.1 Общие принципы реализации

Компоненты Master и Agent реализованы как ASP.NET Core Minimal API на .NET 8. Библиотека Chronos.Core публикуется как отдельный пакет и переиспользуется в консольных сценариях, тестах и серверах. Конфигурация задаётся переменными окружения и секциями appsettings, что упрощает контейнерный деплой.

4.2 Точка входа Master

В Program.cs мастер-узла настраиваются: подключение к PostgreSQL и автоматические миграции EF Core, регистрация DbContextFactory, HTTP-клиенты для вызовов агентов, endpoint-ы Swagger, статические файлы UI, фоновые службы (очистка устаревших агентов и аудита, репликация томов при наличии S3, генерация HAProxy при включённом каталоге динамической конфигурации).

4.3 Точка входа Agent

Агент регистрирует маршруты из AgentRoutes и AgentMainRoutes, поднимает планировщик проверок, при необходимости — встроенный MinIO или внешнее объектное хранилище для сценария прямой выгрузки снимков. Параметры таймаутов и параллелизма тестов задаются конфигурацией, чтобы не перегружать слабые стенды.

4.4 Реализация подсистемы TCP-маршрутизации

Реестр маршрутов сохраняет JSON и генерирует фрагмент chronos-tcp.cfg. Запись выполняется во временный файл с последующим переименованием; кодировка — UTF-8 без BOM (критично для парсера HAProxy). Валидация listen-порта и диапазона публикуемых портов выполняется до изменения файлов.

Пример сгенерированного фрагмента:

Листинг 3: Фрагмент listen-секции HAProxy

```
listen chronos_tcp_идентификатор<>
  bind *:<listenпорт->
  mode tcp
  option tcplog
  server backend_идентификатор<> host.docker.internalпорт:<> check
```

4.5 Алгоритм реестра TCP-маршрутов

Подсистема на стороне Master реализует цикл: валидация входных данных (диапазон listen-порта, отсутствие конфликта с уже зарегистрированными маршрутами и зарезервированными портами) → обновление JSON-реестра в каталоге динамической конфигурации → генерация текстового фрагмента `chronos-tcp.cfg`. Запись на диск выполняется через временный файл и переименование, чтобы внешний процесс NProxy не прочитал частично записанный файл.

Выделение listen-порта при автоматическом режиме сводится к перебору кандидатов внутри заданного MIN..MAX с пропуском занятых значений; предпочтительный порт (если передан) проверяется первым. При удалении маршрута реестр и конфигурация приводятся в согласованное состояние за одну операцию; UI отображает результат после повторного чтения с сервера.

4.6 Фоновая репликация томов

Служба `VolumeReplicationHostedService` выполняется на лидирующем экземпляре Master (в MVP это единственный экземпляр). Цикл обхода политик: чтение включённых политик из БД → при необходимости запрос свободного места на диске целевого агента → листинг объектов в S3 по префиксу и `group` по `MaxCopies` → принятие решения о новом снимке с учётом интервала и объёма предыдущей передачи → HTTP-вызов агента на `endpoint` снимка тома. Длительная работа вынесена в фон, чтобы не блокировать основной поток обработки API.

4.7 Проксирование запросов UI к агентам

Часть страниц интерфейса запрашивает данные напрямую с узла агента (логи, граф сервисов, диагностика). Master выступает обратным прокси: по

идентификатору агента из БД выбирается `baseUrl`, формируется исходящий HTTP-запрос с тем же путём и методом (с ограничениями безопасности), ответ транслируется клиенту. Таким образом, браузер пользователя не должен иметь прямого сетевого доступа ко всем агентам — достаточно доверия к Master.

4.8 Сводная таблица групп UI API

Таблица 2 не дублирует полный OpenAPI, а задаёт ориентиры для чтения кода `UiV1Extensions` и связанных обработчиков.

Таблица 2: Группы endpoint-ов UI API Master (обзор)

Группа	Назначение
Агенты	Список зарегистрированных узлов, детали, действия администратора, связанные с доступностью.
Проекты	Обзор размещений, переход к логам и управлению стеком на выбранном агенте.
TCP-маршруты	CRUD маршрутов, просмотр сгенерированного фрагмента конфигурации, диагностика конфликтов портов.
Архив	Просмотр архивных метаданных проектов, восстановление в работу при поддержке сценария.
Песочница	Безопасный предпросмотр Fluent/YAML без записи в продуктивные сущности (при включении функции).
Служебные	Здоровье сервиса, версии, вспомогательные данные для отладки UI.

4.9 Контейнеризация стенда

Файл `deploy/docker/docker-compose.yml` поднимает PostgreSQL (с специализацией БД), Nginx со скриптом динамического reload, Loki, Grafana, образы Master и Agent. Такой стенд использовался при функциональных испытаниях.

4.10 Клиентская часть

Single Page Application собрана в `Chronos.Master.Web` (Vite, React, TypeScript, маршрутизация `react-router-dom`). Оболочка `ShellLayout` задаёт шапку с логотипом и меню переходов: дашборд, агенты, проекты, архив проектов,

TCP routing, sandbox. Запросы к данным агентов выполняются через UI API Master (`/api/v1/...`), который проксирует вызовы к нужному Chronos.Agent; для части обзорных данных (например, списка агентов) допускаются прямые запросы к маршрутам Master, если они открыты с того же origin.

В подразделе ниже вынесены макеты под скриншоты каждого основного экрана: при подготовке итогового PDF блок с серой рамкой заменяется файлом `.png/.pdf` (командой `\includegraphics`), размещённым, например, в каталоге `figures/` рядом с `main.tex`.

4.11 Экраны веб-интерфейса Chronos.Master.Web

Ниже приведены скриншоты экранов в соответствии с маршрутами React (`App.tsx`). На рисунки даны отсылки в тексте главы 3 и настоящей главы.

Скриншот дашборда (`/ui/dashboard`) не приложен: добавьте фото/`ui.dashboard/1.jpg` при наличии кадра.

Рис. 4.1. Главная панель (дашборд) веб-интерфейса

Скриншот списка агентов (`/ui/agents`) не приложен: добавьте фото/`ui.agents/1.jpg` при наличии кадра.

Рис. 4.2. Список агентных узлов

4.12 Практическая часть: запуск из кода

Практическая демонстрация возможностей Chronos.Core заключается в программном создании экземпляра `ComposeBuilder`, описании сервисов, портов и томов, вызове валидации и методов `PublishRemoteAsync/StartRemoteAsync` (или локального `StartAsync`) с указанием URL агента и ключа API.

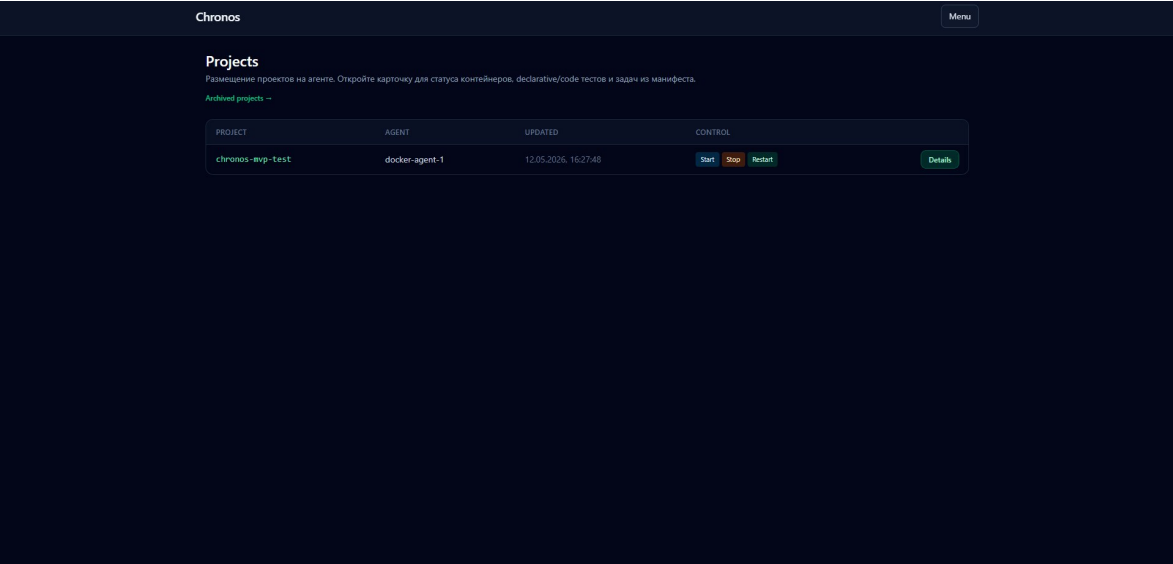


Рис. 4.3. Список compose-проектов кластера

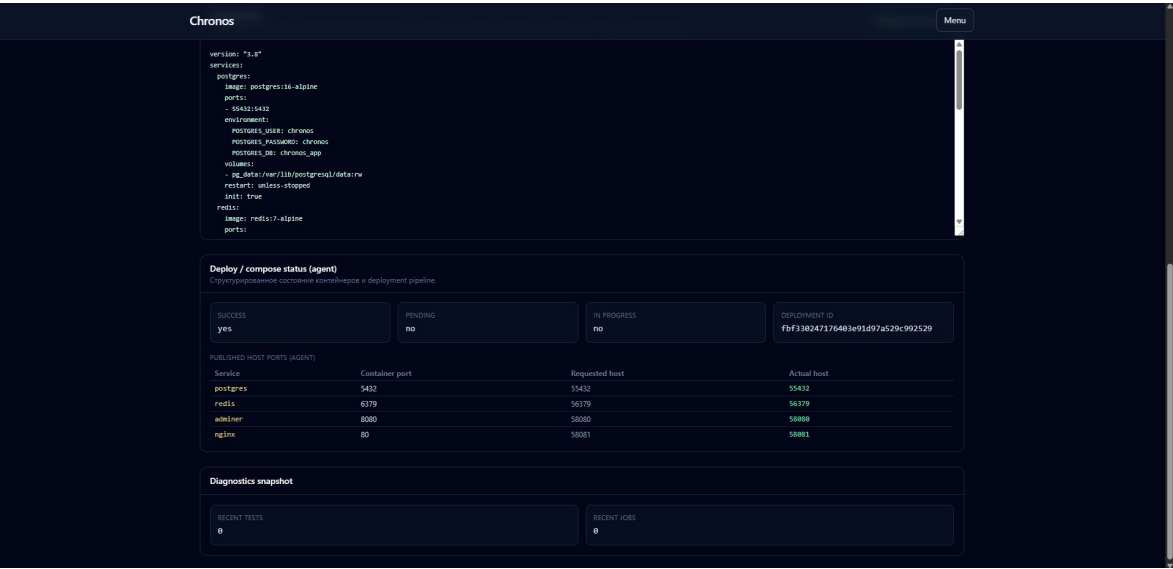
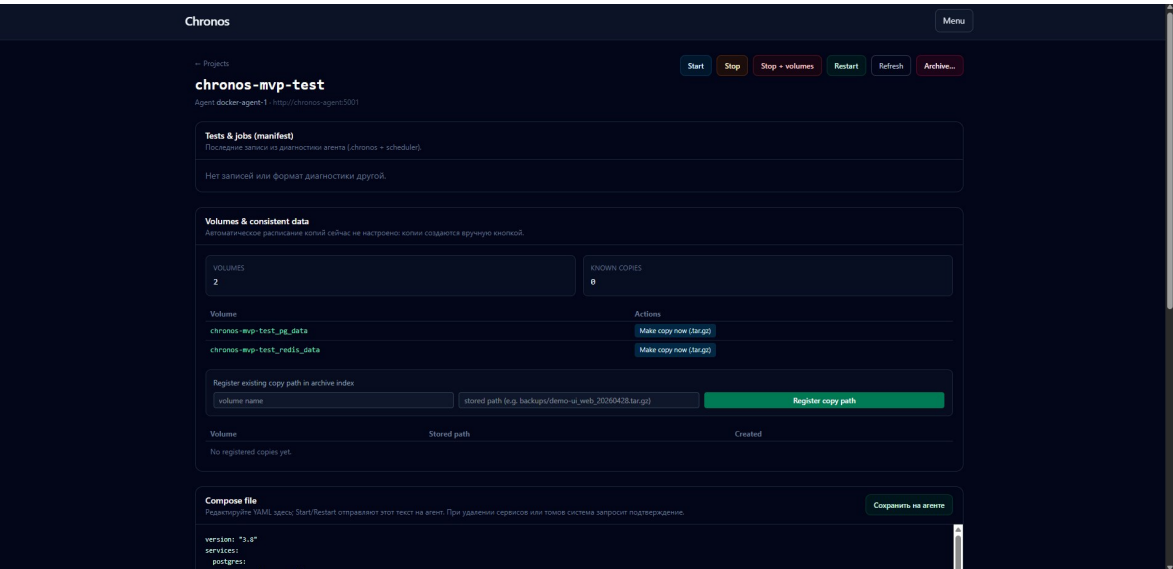


Рис. 4.4. Детальная страница проекта

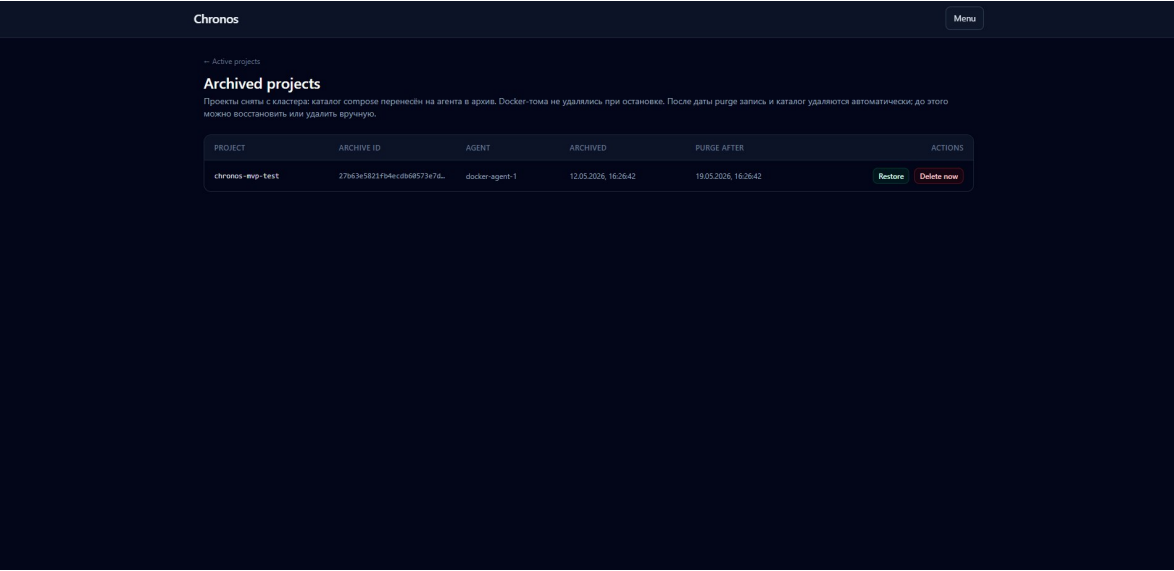


Рис. 4.5. Архивированные проекты

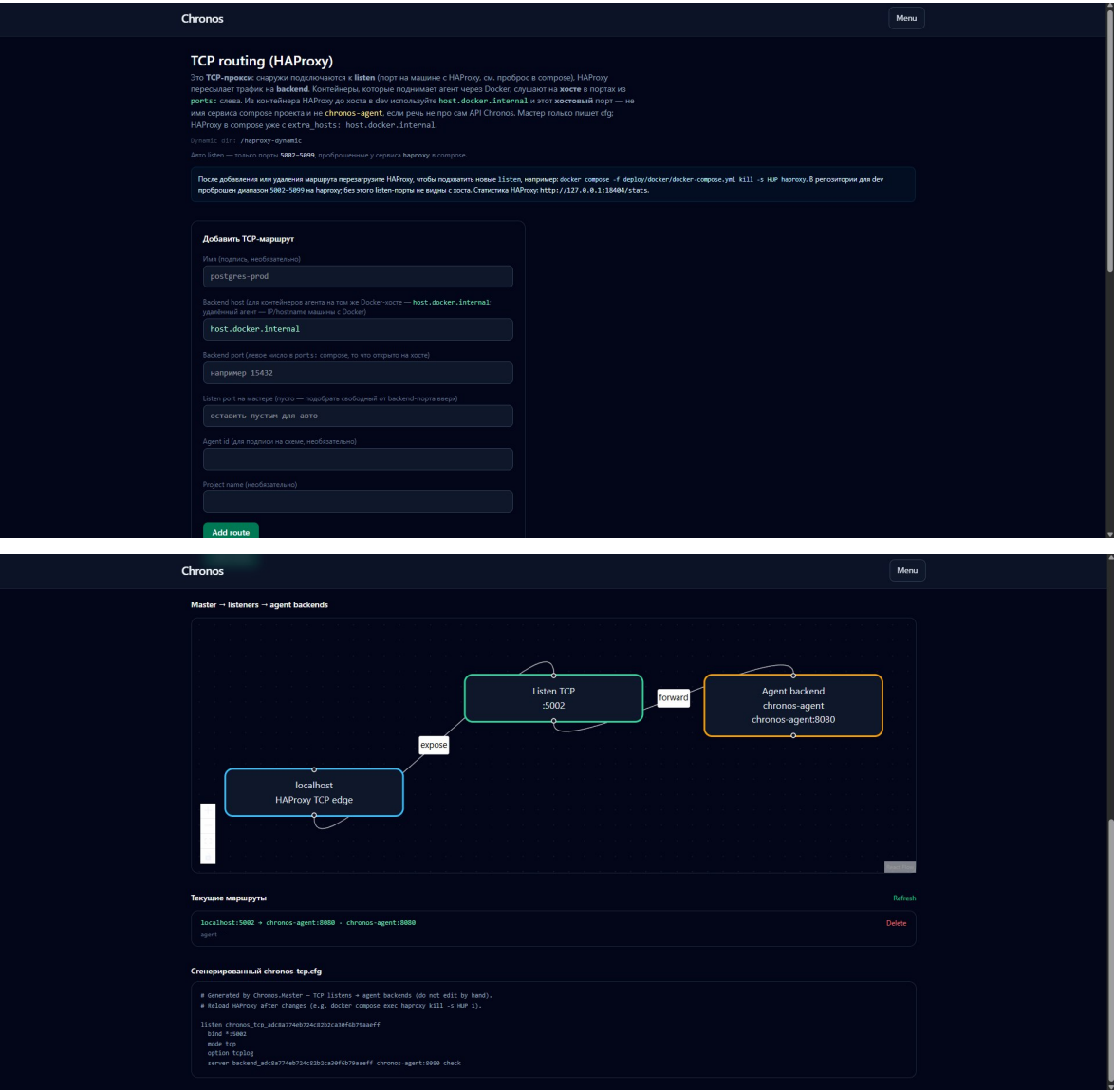


Рис. 4.6. Управление TCP-маршрутизацией (HAProxy)

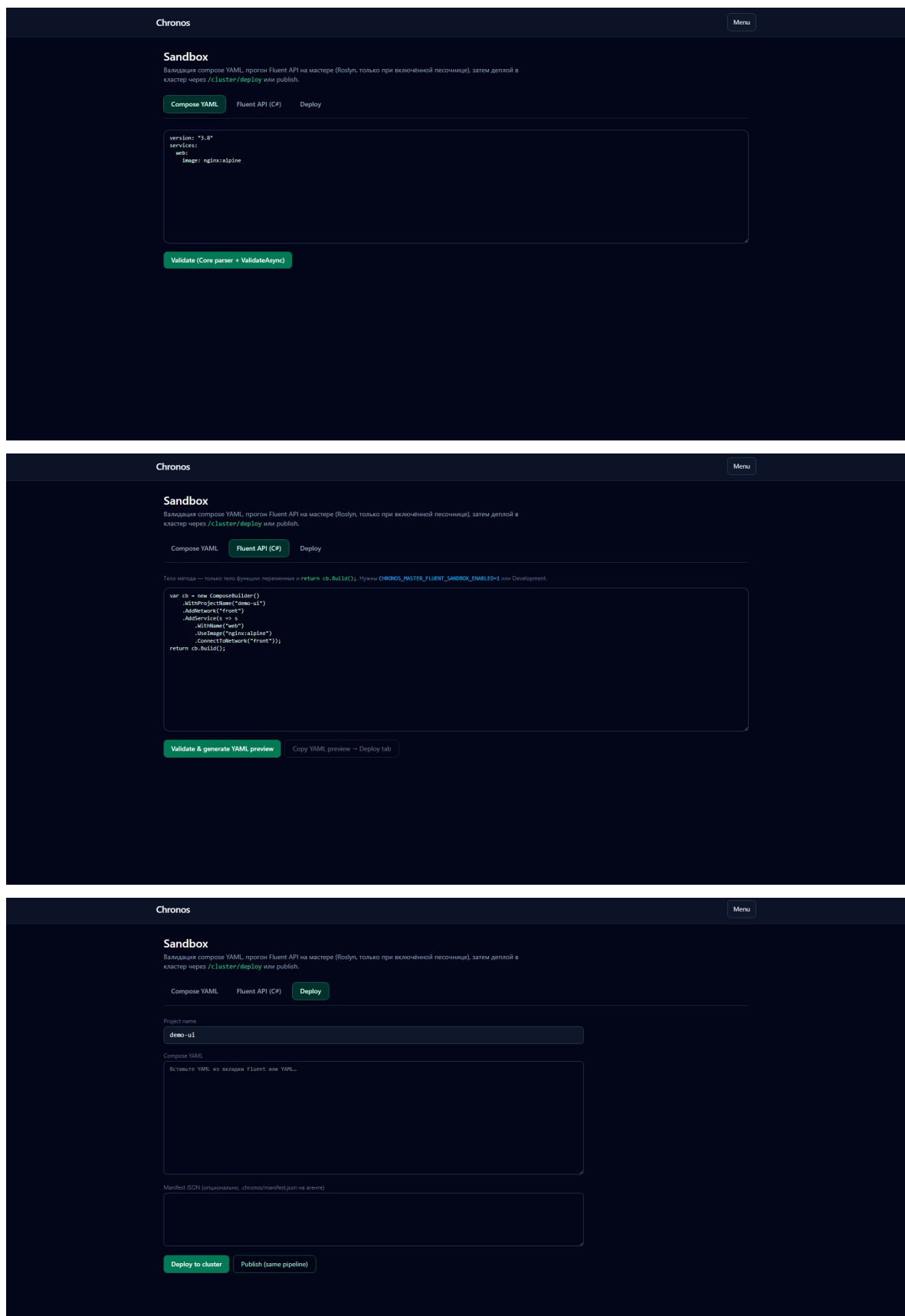


Рис. 4.7. Песочница YAML и Fluent API

В данной версии текста ВКР листинги конкретного кода автора (фрагменты из репозитория или учебного примера) вставляются в этот подраз-

дел вручную по согласованию с научным руководителем, чтобы сохранить актуальность строк и номеров файлов. Рекомендуется включить:

- минимальный пример построения одного сервиса с публикацией порта;
- пример добавления артефакта деплоя;
- пример вызова удалённого старта после PublishAsync.

4.13 Безопасность

Реализованы проверки API-ключей, ограничения на опасные команды, редактирование потенциально чувствительных подстрок в логах. Для production-развёртывания рекомендуется дополнительно вынести секреты в внешние хранилища и ограничить сетевой доступ к `docker.sock` на хосте агента.

5 Экспериментальная проверка и оценка результатов

5.1 Методика испытаний

Проверка системы проводилась на локальном стенде, поднимаемом из `deploy/docker/docker-compose.yml`. Перед началом серии тестов фиксировались версии образов (Master, Agent, PostgreSQL, HAProxy, Loki, Grafana) и параметры переменных окружения, влияющих на диапазон TCP-портов и пути динамической конфигурации HAProxy.

Сценарии. Набор проверок согласован с каталогом функциональных сценариев в приложении Б: регистрация и heartbeat агента, публикация и жизненный цикл compose-проекта, добавление и удаление TCP-маршрутов (в том числе с автоматическим listen), перезагрузка HAProxy после изменения файла, отказ при конфликте порта, просмотр сгенерированного конфига в UI. Для каждого сценария фиксировались: входные данные, ожидаемый результат по критериям ниже, фактический статус HTTP и наблюдения в логах Loki.

Контроль среды. Отдельно проверялась согласованность сетевой модели Docker: доступность `host.docker.internal` из контейнера HAProxy к порту, опубликованному compose на хосте агента. На Linux-хостах без Docker Desktop эквивалентная схема может потребовать иной адресации backend — это зафиксировано как ограничение стенда, а не как дефект кода при условии корректной настройки.

Воспроизводимость. Каждый ключевой сценарий выполнялся минимум дважды: «холодный» старт после `docker compose up` и «тёплый» повтор с уже заполненной БД и существующими маршрутами. Это выявляет ошибки инициализации и ошибки, связанные с накопленным состоянием.

5.2 Критерии успешности

Сценарий считается успешным, если выполняются условия:

- API возвращает консистентный статус и понятные диагностические сообщения;
- в UI отображается фактическое состояние маршрутов;

- сгенерированный `chronos-tcp.cfg` проходит валидацию HAProxy;
- TCP-подключения по `listen`-порту доходят до целевого `backend`-порта;
- перезапуск компонентов не приводит к потере критического состояния.

5.3 Результаты функциональных испытаний

В результате испытаний подтверждено:

- корректное добавление маршрутов с автоматическим выбором `listen`-порта в допустимом диапазоне;
- корректное удаление маршрутов и обновление конфигурации прокси;
- стабильная работа сценария проксирования к `host.docker.internal:PORT`;
- успешное отображение маршрутов и их состояния в UI.

Соответствие каталогу сценариев. Сценарии F-01–F-04 (регистрация, heartbeat, публикация, запуск) и F-07–F-14 (TCP-маршруты, reload HAProxy, доступ к backend, просмотр `cfg`) на стенде автора выполнены успешно. Сценарии F-15–F-18 (песочница, очистка агентов, аудит, повторный подъём) проверялись выборочно: подтверждена работоспособность путей, критичных для демонстрации дипломного прототипа; полный регрессионный прогон рекомендуется вынести в отдельный контур CI.

Выявленные и устраненные в процессе реализации дефекты:

- ошибка HAProxy при наличии UTF-8 BOM в начале динамического конфига;
- проблемы запуска shell-скрипта перезагрузки при CRLF-окончаниях строк;
- неоднозначность пользовательской интерпретации `backend`-портов (порт контейнера vs порт хоста).

5.4 Ограничения эксперимента

Испытания носят функциональный характер: не измерялись предельная пропускная способность ТСП-прокси, задержки при одновременной публикации десятков проектов и деградация БД при длительном накоплении аудита. Нагрузочное тестирование и формализованный отчет SLA не входили в объем работы и относятся к направлениям развития.

5.5 Обсуждение результатов

Полученные результаты подтверждают рабочую гипотезу: для существенной доли практических задач возможно построить инженерно пригодную оркестрацию на основе Compose без перехода на тяжелые кластерные платформы. Ключевым фактором является наличие централизующего слоя (Master + UI) и стандартизованного агентного контура.

Система показала устойчивость в условиях частых изменений маршрутов и конфигураций. Наиболее критичные риски концентрируются в эксплуатационной плоскости:

- ошибки в сетевой адресации backend;
- неконсистентность окружения (различия Docker Desktop и Linux host-mode);
- неверная трактовка опубликованных портов.

5.6 Направления дальнейшего развития

В рамках следующих итераций целесообразно реализовать:

1. multi-master режим с отказоустойчивым лидерством;
2. расширенную модель авторизации и ролей;
3. автоматический discovery опубликованных портов compose-проектов агента;
4. формализованные нагрузочные бенчмарки и тесты деградации;
5. расширенную интеграцию с внешними системами наблюдаемости.

6 Заключение

В работе спроектирован и реализован программный комплекс **Chronos** — платформа для централизованного управления Docker Compose-проектами на нескольких узлах с веб-интерфейсом, динамической TCP-маршрутизацией через HAProxy, политиками резервного копирования томов в объектное хранилище и централизованным сбором логов в связке Loki/Grafana.

Достигнуты следующие результаты:

- обоснована актуальность подхода «compose + управляющий контур» на фоне Kubernetes и классических панелей управления Docker;
- сформулированы и реализованы требования к модульной архитектуре (Core, Agent, Master, Web);
- реализованы публикация проектов, удалённое управление жизненным циклом, регистрация агентов, декларативные проверки и артефакты деплоя;
- внедрены механизмы снимков томов, фоновой репликации по политикам Master и интеграции с S3-совместимым API;
- реализован реестр TCP-маршрутов с генерацией конфигурации HAProxy и сценарием динамического применения на стенде;
- подтверждена работоспособность решения на локальном docker-compose стенде.

Практический итог — работающий комплекс, который **уже применяется автором локально** для управления собственной инфраструктурой разработки: развёртывание сервисов, маршрутизация портов и наблюдаемость сведены к единому контуру без обязательного перехода на Kubernetes.

Направления развития включают усиление отказоустойчивости Master, расширение модели доступа, автоматизацию подсказок по портам из состояния compose и расширенные нагрузочные испытания.

Цель работы достигнута, поставленные задачи решены.

А Приложение А. Фрагменты конфигураций

А.1 Фрагмент docker-compose стенда

Листинг 4: Ключевые части deploy/docker/docker-compose.yml

```
services:
  haproxy:
    image: haproxy:3.0-alpine
    entrypoint: ["/bin/sh", "/usr/local/etc/haproxy/run-with-dynamic-reload.sh"]
    extra_hosts:
      - "host.docker.internal:host-gateway"
    ports:
      - "18404:8404"
      - "5002-5099:5002-5099"

  chronos-master:
    environment:
      CHRONOS_HAPROXY_DYNAMIC_DIR: /haproxy-dynamic
      CHRONOS_HAPROXY_LISTEN_PORT_MIN: "5002"
      CHRONOS_HAPROXY_LISTEN_PORT_MAX: "5099"
      CHRONOS_HAPROXY_TCP_SUGGESTED_BACKEND_HOST: host.docker.internal
```

А.2 Фрагмент алгоритма выбора listen-порта

Листинг 5: Логика выделения listen-порта

```
internal static int? Allocate(int preferred, HashSet<int> reservedByRoutes,
    int maxScan, int? listenMin, int? listenMax)
{
    // 1) нормализация диапазона
    // 2) перебор candidate = preferred + i
    // 3) пропуск занятых или невалидных портов
    // 4) возврат первого подходящего порта
}
```

В Приложение Б. Каталог сценариев и эксплуатационные материалы

В.1 Каталог функциональных сценариев

ID	Сценарий	Критерий приемки
F-01	Регистрация агента	После старта Agent появляется в списке Master с валидным agentId и baseUrl.
F-02	Heartbeat агента	Поле последнего heartbeat обновляется периодически, агент не удаляется TTL-процедурой.
F-03	Публикация compose-проекта	API возвращает успешный статус, на диске агента сохраняется compose-файл проекта.
F-04	Запуск проекта	После команды start сервисы переходят в состояние running, статус доступен через API.
F-05	Остановка проекта	После stop сервисы удаляются/останавливаются в соответствии с настройками.
F-06	Чтение логов	Запрос логов возвращает поток строк по выбранному сервису без критических ошибок.
F-07	Добавление TCP-маршрута (auto)	При пустом listen выбирается первый доступный порт в диапазоне master-конфига.
F-08	Добавление TCP-маршрута (fixed)	При явном listen в допустимом диапазоне маршрут сохраняется и отображается в UI.
F-09	Отказ при конфликте listen-порта	API возвращает информативную ошибку, состояние реестра не изменяется.
F-10	Удаление TCP-маршрута	Маршрут удаляется из JSON и cfg, в UI больше не отображается.
F-11	Релоад HAProxy после изменения cfg	Изменение файла приводит к HUP-сигналу и применению новых listen без ручного вмешательства.
F-12	Проверка защиты BOM	При обнаружении BOM в cfg конфигурация переписывается в UTF-8 без BOM.

ID	Сценарий		Критерий приемки
F-13	Доступ к backend на хосте		Маршрут <code>host.docker.internal:PORT</code> проходит health check и пропускает TCP-трафик.
F-14	Просмотр сгенерированного cfg в UI		Пользователь видит актуальный текст динамического конфига для диагностики.
F-15	Sandbox preview	Fluent	Код в sandbox формирует YAML или возвращает контролируемую ошибку валидации.
F-16	Очистка stale agents		Фоновая задача удаляет неактивных агентов по заданному TTL.
F-17	Аудит операций		Критичные действия фиксируются в журнале аудита в базе Master.
F-18	Повторный подъем стека		После рестарта compose состояние маршрутов и БД консистентно восстанавливается.

В.2 Набор эксплуатационных рисков и мер снижения

Риск	Вероятное проявление		Мера снижения
Несоответствие backend host топологии	TCP health check в HAProxy в состоянии DOWN		Использование <code>host.docker.internal</code> для хостовых портов, явная документация в UI.
Конфликт портов	listen- Отказ добавления маршрута или bind error при старте HAProxy		Диапазон listen в env, проверка занятости и резервирования портов в реестре.
CRLF в shell-скриптах	Ошибки illegal option при запуске entrypoint		Правило <code>.gitattributes</code> и контроль формата файлов в CI.

Риск	Вероятное проявление	Мера снижения
ВОМ в динамическом cfg	Ошибка парсинга HAProxy unknown keyword на первой строке	Принудительная запись UTF-8 without ВОМ и автоисправление при загрузке.
Некорректный HUP/reload	Маршрут добавлен в файл, но не применен процессом	Автоматический watcher-скрипт и явные health checks.
Проблемы доступности БД	Ошибки при старте Master/Agent или миграциях	Health checks postgres, отложенный запуск сервисов по depends_on.
Рост аудита без контроля retention	Деградация БД по размеру таблиц	Периодическая очистка старых записей по configurable retention.
Неоднозначность user input в UI	Пользователь указывает контейнерный порт вместо хостового и наоборот	Контекстные подсказки и информативные ошибки API.

В.3 Расширенный шаблон экспериментальной главы

Ниже приведен текстовый шаблон, который может быть использован для расширения основной главы результатов до полного стандарта кафедральной ВКР. Каждый подраздел рекомендуется дополнять таблицами замеров, скриншотами и логами запусков.

1. Описание аппаратно-программного стенда. Указать характеристики хоста (CPU, RAM, диск), версию операционной системы, версии Docker Engine, Docker Compose, .NET SDK/runtime, браузера для UI-тестов и сетевые ограничения. Важно фиксировать точные версии образов (postgres:16-alpine, haproxy:3.0-alpine, grafana/grafana:11.3.0, grafana/loki:3.1.1), поскольку различия в минорных релизах влияют на воспроизводимость.

2. Методика проведения измерений. Определить повторяемые процедуры: число прогонов каждого сценария, метод фиксации времени (напри-

мер, timestamp логов), критерии успешности и правила отброса выбросов. Для корректного сравнения важно разделять “холодный” запуск (первый старт после поднятия образов) и “теплый” запуск (повторный запуск без пересборки).

3. Серия экспериментов по маршрутизации. Эксперименты рекомендуется разделить на: а) добавление 1 маршрута; б) добавление серии маршрутов; в) удаление и повторное добавление; г) сценарий конфликтующих listen-портов; д) сценарий недоступного backend. Для каждого эксперимента фиксировать: время отклика API, время применения конфигурации HAProxy, фактическую доступность конечного backend.

4. Эксперименты устойчивости. Проверить поведение при рестарте контейнера HAProxy, Master и Agent, а также при рестарте всего compose-стека. Ключевая проверка: после восстановления маршруты должны остаться консистентными в JSON-реестре, UI и runtime-состоянии HAProxy.

5. Эксперименты ошибок ввода. Проверить реакции на пустой backend host, нечисловой backend port, порт вне диапазона 1–65535, listen-порт вне опубликованного диапазона, дублирование listen-порта. Для диплома важно показать не только “happy path”, но и инженерную зрелость обработки ошибок.

6. Анализ результатов. Сформулировать выводы: какие сценарии устойчивы, где система требует операторской квалификации, какие ошибки перехватываются автоматически, а какие остаются зоной ответственности инженера эксплуатации.

В.4 Таблица API-эндпоинтов для документирования

Путь	Метод	Подсистема	Назначение
/agents/register	POST	Master API	Регистрация агентного узла в реестре Master.
/agents/{id}/heartbeat	POST	Master API	Обновление heartbeat агента и метрик состояния.
/agents	GET	Master API	Получение списка активных и исторических агентных узлов.
/api/v1/haproxy/tcp-routes	GET	UI API	Чтение списка маршрутов и сгенерированного HAProxy cfg.

Путь	Метод	Подсистема	Назначение
/api/v1/haproxy/ tcp-routes	POST	UI API	Добавление TCP-маршрута с валидацией listen/backend параметров.
/api/v1/haproxy/ tcp-routes/{id}	DELETE	UI API	Удаление маршрута по идентификатору.
/api/v1/sandbox/ fluent-preview	POST	UI API	Выполнение sandbox-просмотра Fluent-кода и генерация YAML.
/projects	GET	Agent API	Список compose-проектов на агентном узле.
/projects/{name}/ status	GET	Agent API	Статус конкретного проекта и контейнеров.
/projects/{name}/ logs	GET	Agent API	Логи сервисов compose-проекта.